

1. Objetivo General

- Desarrollar una aplicación que permita reafirmar el conocimiento de los **paradigmas de programación imperativo y orientado a objetos**.

2. Objetivos Específicos

- Desarrollar una aplicación en el lenguaje de programación C y Java.
- Aplicar los conceptos de programación imperativa y orientada a objetos.
- Crear y manipular listas como estructuras de datos.

3. Datos Generales

- El valor de la tarea: 15% (7.5% C y 7.5% Java) + 5% Anexo de Atributos.
- Nombre código: spa**CE**invaders.
- La tarea debe ser implementada en grupos de no más de 3 personas.
- La **fecha de entrega** es 11/Junio/2026.
- Cualquier indicio de copia será calificado con una nota de 0 y será procesado de acuerdo al reglamento.

4. Descripción del juego.

Space Invaders es uno de los videojuegos más importantes de la historia. Su objetivo es eliminar oleadas de alienígenas con un cañón láser y obtener la mayor cantidad de puntos posible.

El jugador controla un cañón que puede moverse a la derecha o izquierda y un botón de disparo. Tiene que ir destruyendo los extraterrestres invasores (de los cuales hay tres tipos: con forma de calamar otorga 10 pts en caso de ser derribado, de cangrejo otorga 20pts en caso de ser derribado y de pulpo otorga 40 pts en case de ser derribado) que van acercándose a la tierra cada vez más rápidamente a medida que el jugador va destruyendo a los enemigos. Este ciclo se puede repetir en forma indefinida. Si los invasores llegan al cañón controlado por el jugador, el juego termina (Wikipedia, 2026).

Cada cierto tiempo aparece en la pantalla, por encima de los invasores, un platillo volador que se mueve aleatoriamente de derecha a izquierda o de izquierda a derecha y que no agrega una puntuación definida, sino puntos extras en cantidades aleatorias. Además, se tienen cuatro escudos de protección terrestre (más parecidos a bunkers) que cubren al jugador del fuego alienígena, pero que son destruidos gradualmente por los disparos de los invasores y el cañón del jugador (Wikipedia, 2026).

Pueden jugarlo en línea para que se den una mejor idea en (Pacxon, 2026).



- 4.1. **El servidor:** Deberá ser implementado en **Java**. Mantendrá **TODA** la lógica del juego. Será el encargado de crear y destruir los extraterrestres en las posiciones que el administrador lo decida en cualquier momento de la partida, llevará la puntuación, las vidas restantes del jugador (iniciara con 3 vidas y cada vez que lo impacte una bala se le disminuye en uno la vida) y crear los ovnis con el valor aleatorio (introducido por el usuario), velocidad de desplazamiento de los extraterrestres y el estado de los bunkers.

Todas estas variables podrán ser modificadas en tiempo de ejecución a través de un mensaje que se le envía al cliente.

Cada vez que el jugador elimine todos los extraterrestres se le asignará una vida al jugador y el juego reinicia con una velocidad de desplazamiento de los extraterrestres Mayor a la anterior.

4.1.1. Mensajes a manejar:

- 4.1.1.1. Crear Extraterrestre (X, Y, Pts). Ejemplo Crear (1,1, 1000)
- 4.1.1.2. Crear OVNI dirección puntos. Ejemplo: OVNI I-D 1500.
- 4.1.1.3. Aumentar Velocidad de desplazamiento de los extraterrestres. Ejemplo: Velocidad 100
- 4.1.1.4. Estado de Bunkers: Ejemplo Bunkers 70%, Bunkers 40%, Bunkers 0%

El cliente Jugador se refiere a la interfaz gráfica del juego debe realizarse **en C**, controla el cañón que utiliza el usuario, interpreta la estructura enviada por el servidor y le informa al servidor cuando un extraterrestre es eliminado a través de un mensaje al servidor.

Para revisión como mínimo el servidor debe soportar al menos 2 clientes jugadores.

- 4.2. **El cliente espectador:** Se podrá unir a una partida existente pero sólo podrá observar lo que

sucede.

4.3. **Conexión cliente servidor:** La conexión entre el servidor y los clientes debe realizarse utilizando Sockets.

4.4. **Aspectos de implementación en C:** Recuerden se evaluará que todas las constantes estén en un archivo aparte. Se evaluará el uso de structs.

Se requiere un ejecutable para la revisión.

4.5. **Aspectos de implementación en Java:** Recuerden se evaluará que todo este programado según los conceptos de OO (Clases, paquetes, **patrones** al menos deben implementar 2 patrones, se sugieren el Adapter, Observador y Abstract Factory o Factory -Singleton no cuenta como patrón para revisión-).

Se requiere un ejecutable para la revisión.

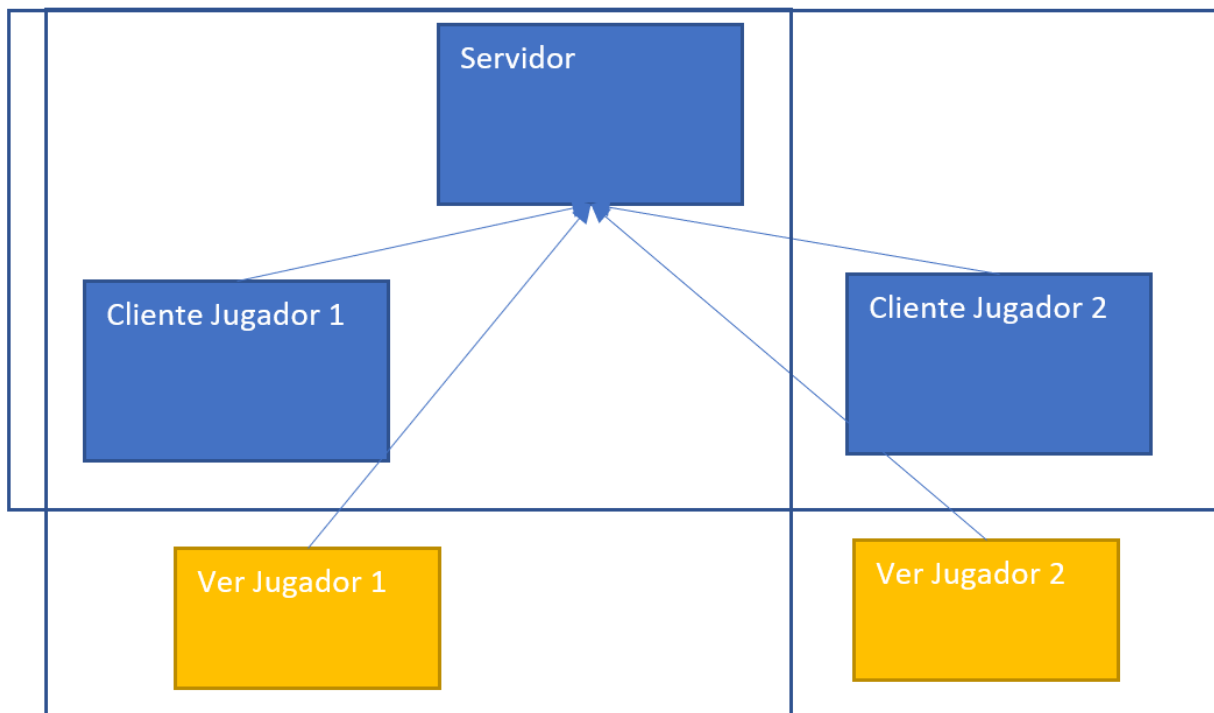
4.6. **Control:** Se debe implementar un control físico de dos botones para el manejo del cañón. Se solicita utilizar un Raspberry PI Pico su programación debe realizarse en C. Su comportamiento debe ser:

4.6.1. Un botón controlara el cañón, un toque al botón significa moverse a la izquierda, dos toques a la derecha.

4.6.2. El segundo botón será el botón de disparo.

4.6.3. El control debe comunicarse por medio de UART o I2C.

4.7. **Se requiere un ejecutable para la revisión.**



5. Entregables

5.1 Código fuente comentado.

5.2 Manual de usuario.

6. Documentación

1. Se deberá entregar un documento que contenga:
 - 1.1. Manual de usuario: cómo ejecutar el programa.
 - 1.2. Descripción de la utilización de las estructuras de datos desarrolladas (por ejemplo descripción del nodo de una lista).**
 - 1.3. Descripción detallada de los algoritmos desarrollados.**
 - 1.4. Problemas sin solución: En esta sección se detalla cualquier problema que no se ha podido solucionar en el trabajo.
 - 1.5. Plan de Actividades realizadas por estudiante: Este es un planeamiento de las actividades que se realizaran para completar la tarea, este debe incluir descripción de la tarea, tiempo estimado de completitud, responsable a cargo y fecha de entrega.
 - 1.6. Problemas encontrados: descripción detallada, intentos de solución sin éxito, soluciones encontradas con su descripción detallada, recomendaciones, conclusiones y bibliografía consultada para este problema específico.
 - 1.7. Conclusiones del proyecto.
 - 1.8. Recomendaciones del proyecto.
 - 1.9. Bibliografía consultada en todo el proyecto
2. Bitácora en digital, donde se describen las actividades realizadas, desde reuniones con el compañero de trabajo, investigaciones, consultas, etc. Está se puede encontrar hecha a mano, se debe describir todo por más insignificante que sea, esto demostrará si ustedes están trabajando en realidad. Este es su diario de trabajo, llevan seguimiento de todo en el tiempo, imaginen que si un compañero los releva en su trabajo, le bastaría con leer sus bitácoras para seguir el trabajo.

7. Evaluación

1. El proyecto tendrá un valor de un 70% de la nota final, debe estar funcional.
2. La documentación tendrá un valor de un 20% de la nota final, cumplir con los requerimientos especificados en la documentación no significa que se tienen todos los puntos, se evaluará que la documentación sea coherente, acorde al tamaño del proyecto y el trabajo realizado, no escatimen en documentación.
3. La defensa tendrá un valor de 10%, todos los integrantes del grupo deben participar.
4. Cada grupo recibirá una nota en cada uno de los siguientes apartados Código, Documentación y defensa.
5. El profesor no sólo evaluará la funcionalidad del proyecto, esto quiere decir que aunque el proyecto este 100% funcional esto no implica una nota de un 100, ya que se evaluarán aspectos de calidad de código, aplicación del **paradigma imperativo y orientado a objetos**, calidad de documentación interna y externa y trabajo en equipo.
6. No se revisarán funcionalidades parciales, ni funcionalidades no integradas.
7. Es responsabilidad de cada miembro del grupo conocer su código, el profesor puede preguntar a cualquier miembro del grupo que le explique alguna funcionalidad/porción de código.
8. De las notas mencionadas en los puntos 1, 2 y 3 se calculará la Nota Final del Proyecto.

9. Las citas de revisión oficiales serán determinadas por el profesor durante las lecciones o mediante algún medio electrónico.
10. Aun cuando el código, la documentación y la defensa tienen sus notas por separado, se aplican las siguientes restricciones
 - 10.1. Si no se entrega documentación, automáticamente se obtiene una nota de 0.
 - 10.2. Si no se entrega el punto 3 de la documentación se obtiene una nota de 0.
 - 10.3. Si el código y la documentación no se entregan en la fecha indicada se obtiene una nota de 0.
 - 10.4. Si el código no compila se obtendrá una nota de 0, por lo cual se recomienda realizar la defensa con un código funcional.
 - 10.5. Si el grupo no cuenta con los equipos necesarios para realizar la revisión y no avisó al profesor de esta situación obtendrá una nota de 0.
 - 10.6. El código debe ser desarrollado en **el lenguaje de programación C** utilizando el **paradigma de programación imperativo** y java utilizando el **paradigma de programación orientado a objetos**, en caso contrario se obtendrá una nota de 0.
 - 10.7. **NO** presentarse a la defensa se obtendrá una nota de 0.
11. Cada grupo tendrá como máximo 30 minutos para exponer su trabajo al profesor y realizar la defensa de éste, es responsabilidad de los estudiantes mostrar todo el trabajo realizado, por lo cual se recomienda tener todo listo antes de ingresar a la defensa.
12. Cada excepción o error que salga durante la ejecución del proyecto y que se considere debió haber sido contemplada durante el desarrollo del proyecto, se castigará con 2 puntos de la nota final del proyecto.
13. Cada grupo es responsable de llevar los equipos requeridos para la revisión.
14. Durante la revisión únicamente podrán participar los miembros del grupo, asistentes, otros profesores y el coordinador del área.
15. Las revisiones se realizan con los estudiantes matriculados en el curso, cualquier persona fuera de estos y los mencionados en el punto 14, no pueden participar en la revisión.
16. Después de enviada la nota final del proyecto el estudiante tendrá un máximo de 3 días hábiles para presentar un reclamo siempre y cuando la funcionalidad esté completa.

Referencias

Pacxon. (7 de Mayo de 2026). *Space Invaders*. Obtenido de Pacxon:

<http://www.pacxon4u.com/space-invaders/>

Wikipedia. (7 de Mayo de 2026). *Space Invaders*. Obtenido de Wikipedia:

https://es.wikipedia.org/wiki/Space_Invaders